pq402

An API that charges an AI agent, checks it is allowed to buy,
and never learns which buyer it was

Manuel Guilherme Galmanus

5 August 2026

Stellar Summit SP 2026 · Payments and Agent Tooling (SDF DevEx)
sub-lane 3A, Agentic Payments (x402 / MPP)

`https://github.com/Galmanus/pq402`

Abstract

Autonomous software is starting to buy things: data, API calls, compute. The protocols for that are arriving — x402 revives HTTP's long-dormant `402 Payment Required` and settles it on-chain — but they answer only one question, *did the agent pay.* Metered services have a second, *is this agent allowed to buy this at all*, and today's answers to it are an API key or an account, both of which stamp the buyer's identity onto every request they will ever make.

This paper describes a working system that answers both, and answers the second without learning who is asking. The eligibility check is a hash-based zero-knowledge proof judged by a Soroban contract, not by the server; a per-challenge nullifier is burned in contract storage, so single use is enforced by consensus and survives the server being restarted, replaced, or dishonest. In its unlinkable mode the proof publishes a commitment rather than a credential identifier, with fresh blinding on every use: the server learns that *a member of the issuer's set* paid and nothing further, and two payments by one credential share no public value, so they cannot be linked to each other.

Everything reported here runs on Stellar testnet and every claim is backed by a transaction hash. We also record four undocumented details of the x402 protocol on Stellar that each cost a debugging round to find, and publish the fix as a package so the next team does not pay for them again. The limits are stated with the same precision as the results: the proof system is post-quantum, the transaction carrying it is signed with Ed25519 and is not.

---

1. The gap

An agent that buys something must be authorised to buy it. The two mechanisms in use today both work by identifying the buyer.

An **API key** is a bearer identifier. Every call carries it, so the seller accumulates a complete, joinable history of one buyer's behaviour — which is usually not what either party wanted, merely what was easy to build.

An **account** is worse for the buyer and better for the seller, and has the same shape: eligibility is established by being known.

Neither is a failure of care. There has not been an alternative that a server could check cheaply, that a client could produce quickly, and that did not require the server to be trusted with the answer. What follows is an attempt at one.

1.1 Why now

Three things had to be true at once, and only recently are:

- **A payment rail an agent can drive.** x402 defines a 402 response carrying machine-readable payment requirements, and on Stellar the client signs Soroban *authorization entries* rather than transaction envelopes — so a facilitator assembles the transaction and pays the network fee. An agent needs the asset it intends to spend and no XLM at all.
- **A proof cheap enough to verify inside a smart contract.** Circle STARKs over the Mersenne-31 field are the cheapest hash-based proof system in production. Verification fits an ordinary Soroban contract call.
- **Somewhere to put the single-use guarantee that is not the server.** Contract storage. A nullifier burned there cannot be forgotten by restarting a process.

2. What the system does

A request to a paid route meets two independent gates. Neither is decided by the server, and the server cannot overrule either.

gate	mechanism	who decides
payment	x402 `exact`, USDC over its Soroban Asset Contract	Stellar consensus
credential	hash-based STARK, nullifier burned in storage	Stellar consensus

The flow, in the order it happens:

```
agent -> GET /premium
      <- 402 payment-required header + credential challenge

agent proves eligibility locally, 7 ms
agent -> POST /pq/unlock { proof, publics }
server-> Soroban: verify the proof and burn the nullifier, one transaction
      <- pass, and the transaction hash of the burn

agent -> GET /premium + PAYMENT-SIGNATURE + the pass
server-> facilitator: verify, then settle
      <- 200, the goods, and the settlement hash
```

Two orderings in that sequence are deliberate and worth naming.

The credential is checked before the payment. Charging someone and then refusing them is worse than refusing them, and arranging that is the server's fault, not the protocol's.

Verification precedes settlement. `settle` moves money, so a payload that fails verification must never reach it. The package has a test asserting that a rejected verification leaves `settle` uncalled, because the ordering is the only thing standing between a bad payload and a real transfer.

3. The credential, and what the server learns

3.1 Identifying mode

The relation proves, in zero knowledge, that the caller knows a secret s such that

$$\text{leaf} = \text{trunc}_8\big(\pi(s \,\|\, \text{action})\big), \qquad \text{nullifier} = \text{trunc}_8\big(\pi(s \,\|\, \text{round})\big)$$

share that one secret, where π is the Poseidon2 permutation over Mersenne-31. The AIR constrains the secret cells of the two permutation blocks to be equal in the same row; without that constraint a prover could present one member's leaf beside another member's nullifier.

Both published values are *truncated* to a digest. An earlier version published the full sixteen-limb permutation state, and that was a break rather than a wart: π is a bijection and the context sits public beside it, so a single inversion returned the secret. It was found by audit, demonstrated against a real testnet proof in milliseconds, and fixed by publishing digests — which puts recovery back at $|F|^8 \approx 2^{248}$.

In this mode the leaf is a public input. The server therefore knows *which* credential paid, and two payments by one credential are trivially linked. That is anonymous in the sense of not knowing your name; it is not anonymous in the sense of not knowing which of the thousand you are.

3.2 Unlinkable mode

The second mode closes that. The relation publishes

$$C = \text{compress}(\text{leaf} \,\|\, \text{blinder})$$

with a fresh blinder drawn from system entropy on every use, and never the leaf. Two proofs compose on that shared C:

- the **acting** half proves that a valid credential acted under C, and burns its nullifier;
- the **membership** half proves that C sits under the issuer's published Merkle root, through a private authentication path.

What the server learns is exactly: *someone in the issuer's set paid.* Not which member. And because the blinder is fresh, two payments by one credential share no public value at all, so they cannot be linked to each other either.

Three orderings again, each for a reason:

- the two commitments are compared **before** anything is spent — two proofs are only about one credential if they share C, and checking that is free while the proofs cost two transactions;
- membership is verified **before** the acting half — membership is a read-only call and acting burns a nullifier, so there is no reason to burn one for a caller who was never in the set;
- the claimed root is compared against the **issuer's** — a membership proof is only ever a statement about the tree the prover chose, and without that check anyone can build a tree containing their own leaf.

3.3 The challenge is not the server's to choose

The acting contract derives the expected round from the ledger sequence:

$$\text{epoch} = \left\lfloor \frac{\text{ledger\_sequence}}{720} \right\rfloor$$

and refuses any other. Freshness is therefore guaranteed by consensus rather than asserted by whoever is asking, and one credential acts once per epoch — about an hour at Stellar's close time. A server cannot replay an old challenge at a member, and a member cannot pick a convenient one.

4. Four things about x402 on Stellar that are not written down

Each of these cost a debugging round, and each produced an error message that pointed somewhere other than the cause. They are encoded as behaviour in the published package rather than as advice, because a comment does not stop anyone from getting it wrong.

1. **v2 carries the requirements in a `payment-required` header, not the body.** The body is read only when `x402Version == 1`. A server that publishes perfect requirements as JSON alone is invisible to a v2 client, which fails with *Invalid payment required response* having never looked at them. Every 402 a gated route emits needs the header — including the one meaning "you have a session but still owe payment", which is the one everybody forgets.

2. **The signed payload returns in `PAYMENT-SIGNATURE`**, with `X-PAYMENT` the v1 name. Read both.

3. **`extra.areFeesSponsored` must appear in the requirements** or the Stellar `exact` scheme refuses to build a payment at all. That flag is the facilitator's assertion, so it should be fetched from `/supported` at startup rather than claimed on the facilitator's behalf.

4. **Stellar assets carry seven decimals, not six.** `0.10` is `1000000` base units. The six an EVM habit expects is a tenth of the intended price, settling quietly — the worst kind of bug, because it works.

A fifth, adjacent: the Stellar CLI's `-network` takes a configuration *name* (`testnet`) while x402 takes a CAIP-2 *identifier* (`stellar:testnet`). Feeding one to the other fails with *Invalid name*, four frames from anything that mentions networks.

5. Spending limits that are rules, not promises

A client-side cap — `--max 0.10` on the agent — is a promise the agent makes to itself. An agent that has been prompt-injected, misconfigured, or handed a bad tool description simply stops making it, and nothing outside the agent notices.

`agent-treasury` is the same limit as a rule. It is a Soroban contract holding the agent's funds, enforcing two policies before any token moves:

- a **contract allow-list**, so a stolen policy key cannot be pointed at a token the treasury was never meant to touch;
- a **rolling daily cap**, keyed off ledger time rather than a counter that calling more often can reset.

Both refusals happen in consensus. On testnet: a payment inside policy settles; one over the cap is refused with error `#3`; one naming a token outside the allow-list is refused with `#2`; and the remaining budget is unchanged after both, because a refusal consumes nothing.

The budget is committed *before* the transfer: if the transfer panics the whole invocation reverts, so the ordering cannot leak budget, while committing afterwards would leave a window in which a reentrant call still sees the old total.

6. Measurements

All on Stellar testnet, and each number was measured rather than estimated.

proving, identifying mode (median of 15 runs, process start included)	7 ms
proving, range across those runs	6–11 ms
proof size, identifying mode	79,186 B
proof size, unlinkable mode (acting half)	113,360 B
proof size, unlinkable mode (membership half)	82,765 B
settlement fee, paid by the facilitator	23,073 stroops

The fee row is the one worth pausing on: the account charged the network fee is the facilitator's, not the agent's. That is fee sponsorship working, and it is why an agent wallet needs only the asset it intends to spend.

6.1 Security parameters

FRI soundness is approximately queries $\times \log_2(\text{blowup})$ bits before the proof-of-work grind, and it cannot exceed the field the challenges are drawn from. Both halves of that sentence matter.

The identifying gate originally ran at 40 queries with blowup 2: about 48 bits conjectured, and near 24 against a Grover-assisted adversary, since Fiat-Shamir grinding falls to a square-root search. Twenty-four bits is a demonstration parameter wearing a production label.

Raising it on that path was impossible rather than merely expensive: its challenge field is a degree-3 extension of Mersenne-31, about 93 bits, so 46 quantum bits is that path's ceiling at any configuration. The gate now used draws challenges from QM31, a degree-4 extension with a 124-bit ceiling, and runs at 16 queries with blowup 2^6: roughly 96 conjectured bits, near 48 quantum. The configuration is pinned inside the contract rather than passed by the caller, because whoever picks the security level picks how hard their own proof is to forge.

7. Reusability

The protocol work is published, not merely described.

```
npm install x402-stellar-paywall
```

```
const pay = await expressPaywall({ price: "0.10", payTo: RECIPIENT });
app.get("/weather", pay, (req, res) =>
  res.json({ temp: 24, settled_by: req.x402.transaction }));
```

That is the whole integration. Adding `credential: { ... }` adds the eligibility gate; adding `mode: "crowd"` to it makes that gate unlinkable. Bindings exist for Express, Hono and FastAPI — the three frameworks the sub-lane names — over a framework-free core with no runtime dependencies.

It was proven somewhere other than its birthplace: `examples/second-app` is a weather API with no credential logic in it at all, which declares the package as an ordinary dependency, gates one route with one line, and was paid by an agent on testnet.

8. What is claimed, and what is not

We separate these deliberately, because the value of the measurements above depends entirely on the precision of the claims they support.

8.1 The boundary that matters most

Everything here concerns the *proof system*. The *transaction* carrying a proof is signed with Ed25519, which is elliptic-curve and falls to Shor. An adversary with a quantum computer does

not need to break any relation in this paper: they forge the signature on the envelope and the post-quantum content inside becomes irrelevant. This is a post-quantum component running on a pre-quantum chain, and the missing half is Stellar's to supply, not ours.

8.2 Claimed

- **Post-quantum in kind**: hash-based, no elliptic curves, no trusted setup. Nothing in the proof system falls to Shor.
- **Post-quantum in strength**, at the deployed configuration: roughly 96 conjectured bits, near 48 against a Grover-assisted adversary. Both numbers rest on the standard conjectured FRI estimate rather than a proven bound.
- **Single use by consensus**: the nullifier is burned in contract storage, so it survives a restart of the server. It does not survive a redeploy of the verifier, which starts a fresh set.
- **Unlinkability across uses**: computational, resting on the hiding of the commitment under a random-oracle-style assumption with a 248-bit blinder.

8.3 Not claimed

- No bespoke security proof for the sponge-capacity split or the compression convention. The compression function is a truncated public permutation without feed-forward, which is *provably* not collision resistant; no forgery follows from that in this system as far as our analysis goes, and both statements are recorded rather than one of them.
- The identifying mode does not hide which credential paid. Only the unlinkable mode does.
- The two halves of the unlinkable proof are composed by a shared public value, not fused into one trace, so on-chain they are two transactions.
- Amounts and recipients are in the clear. This hides *who*, not *how much* or *to whom*.

8.4 On priority

We know of no prior x402 payment gated by an unlinkable credential, on Stellar or elsewhere, and no prior STARK verified directly by a Soroban contract: everything verifying zero-knowledge proofs on Soroban today is pairing-based, and RISC Zero reaches Stellar only after a Groth16 wrapper that reintroduces pairings and a trusted setup at the last step. That is a statement about what public search surfaced on 2 August 2026, not a proof of absence. If it is wrong, the correction belongs in the repository and will be put there.

9. Reproducing

```
git clone https://github.com/Galmanus/pq402 && cd pq402
npm install && (cd cli && npm install)
cp .env.example .env
node setup.mjs # generates and funds both accounts
curl -s https://channels.openzeppelin.com/testnet/gen # a facilitator key

./demo.sh # the identifying loop, end to end
./demo-crowd.sh # the unlinkable one, end to end
```

One step cannot be scripted: the Circle testnet USDC faucet is captcha-gated, and `setup.mjs` prints the address to paste there.

Running `demo-crowd.sh` twice within the same hour-long epoch produces a refusal — from the contract, not from the script. That refusal is the single-use guarantee showing its face rather than being asserted.

Source, transaction hashes and the full measurement log: `https://github.com/Galmanus/pq402`. The Circle-STARK library the credential relation is built from is riverrun. MIT licensed.